

DATA SCIENCE 2VG3 Homework 5

Submitted Files

- rigidbody2d_friction.py
- rigidbody2d_friction_extra.py
- rigidbody2d_rolling.py
- verify_hw5.py

All measurements below were regenerated from the current code with:

```
python3 verify_hw5.py
```

The verifier reruns the friction baseline, the extra-credit friction variant, the rolling default case, and the four-body race, then stores the reports in output/.

Exercise 1

I completed `rigidbody2d_friction.py` so the scene now advances rigid bodies with gravity, detects square-square contacts with the provided collision code, applies normal impulses with restitution, applies Coulomb friction in the tangential direction, and records reproducible headless reports. The default inclined-plane test uses:

- angle 13 deg
- $\mu_s = \mu_k = 0.5$
- restitution $e = 0.1$

The baseline inclined-plane summary for that default case ends with a tangential speed of 0.030828, which is small but not exactly zero.

Exercise 2

The critical static-friction coefficient for a 13 deg incline is

```
mu_critical = tan(13 deg) = 0.2308681911
```

2(a) Why the block rocks and creeps

The motion is a solver artifact rather than a real physical prediction. The square is resting on a multi-point contact manifold, but the engine handles contact points one at a time in a discrete-time loop. Each frame gravity adds a small downward velocity, then the contact solver and position correction try to remove the resulting penetration. Because that process is sequential and approximate, the normal and tangential responses do not cancel perfectly. The leftover error appears as tiny rotations, tiny tangential velocities, and a slow drift along the plane even when the static-friction threshold is theoretically high enough to hold the block.

2(b) Baseline friction sweep

I swept the baseline engine with the same value for static and kinetic friction each time. The measured trend is:

μ	Final tangential speed	Average tangential acceleration over last half	Interpretation
0.12	9.669751	2.120449	Strong sliding and clear acceleration
0.18	5.255719	1.219703	Still accelerates down the ramp
0.22	0.785928	0.105896	Below $\tan(\theta)$, so it continues to accelerate
0.24	0.082877	0.003215	Above threshold, but the engine still creeps
0.30	0.052580	-0.001510	Nearly static, with residual numerical drift
0.50	0.030827	0.001717	Very small creep only
0.80	-0.025109	0.001432	Tiny oscillatory drift

This matches the expected qualitative transition. Below μ_{critical} the box accelerates. Just above μ_{critical} the acceleration largely disappears, but the baseline engine still fails to hold an exact static equilibrium.

2(c) Improved solver in rigidbody2d_friction_extra.py

For the extra version I changed the code to match a simple "resting body" strategy rather than trying to solve the static-contact case perfectly every frame.

- I added tangential drift correction that depends on the contact mode.
- If the contact is inside the static friction cone, I apply full tangential position correction for that frame.
- If the body is genuinely sliding, I only apply a small fraction of that tangential correction so the block can still move down the plane.
- I track low linear and angular speeds over multiple consecutive frames.
- Once the block has stayed slow enough for long enough and μ_s is above the theoretical threshold, I zero v_t and ω , mark the body as resting, and turn gravity off for that body.

The key settings in this submission are:

- full static tangential correction factor: 1.0
- sliding tangential correction factor: 0.05

- rest speed threshold: 0.09
- rest angular-speed threshold: 0.08
- required quiet frames before sleep: 6

With those changes, the default extra-credit run settles fully. Its final tangential speed is 0.0, `final_sleeping = true`, and gravity is disabled after it comes to rest. In that default run, the block spends about 95.56% of the recorded frames in the sleeping state.

The sweep for the improved version is:

<code>`mu`</code>	Final tangential speed	Average tangential acceleration over last half	Interpretation
0.12	9.531683	2.074039	Still clearly sliding
0.18	5.074559	1.151960	Still clearly sliding
0.22	0.798056	0.105896	Still subcritical, so it accelerates
0.24	0.000000	0.000000	Settles to rest just above the threshold
0.30	0.000000	0.000000	Resting
0.50	0.000000	0.000000	Resting
0.80	0.000000	0.000000	Resting

So the extra version preserves the low-friction sliding regime while removing the artificial high-friction creep that the baseline solver shows.

Exercise 3

The default scenario in `rigidbody2d_rolling.py` is a hollow cylinder of radius 1.0 on a 13 deg incline. The rolling code uses generic shape collision handling, so the round body is not just animated by a formula; it is advanced by the same contact solver.

For the default hollow-cylinder run:

- analytic exit speed: 3.6738349528
- measured exit speed: 3.6656014919
- percent error: -0.2241108%

That is a close match to the expected rolling result.

Exercise 4: The Race

I ran the four required cases with the same incline and measured the speed at the end of the block.

The expected ordering is:

- sliding particle
- sphere
- solid cylinder
- hollow cylinder

This submission produces exactly that ordering. The measured race results are:

Racer	Measured final speed	Analytic speed	Percent error
Sliding particle	5.190916	5.195587	-0.089906%
Sphere	4.384223	4.391073	-0.155991%
Solid cylinder	4.234995	4.242179	-0.169343%
Hollow cylinder	3.665601	3.673835	-0.224111%

The ordering is what energy conservation predicts:

$$v = \sqrt{2gh / (1 + I/(MR^2))}$$

The slider is fastest because no energy is stored in rotation. Among the rolling cases, a smaller rotational inertia gives a larger translational speed, so the sphere beats the solid cylinder, and the solid cylinder beats the hollow cylinder.

Verification Summary

The current verification output reports:

- baseline final tangential speed: 0.0308280382
- extra final tangential speed: 0.0
- extra final sleeping state: true
- rolling default percent error: -0.2241108%
- race order: slider > sphere > solid > hollow

These results answer all parts of the handout with code-generated measurements from the current submission.